FIT5201 - Machine Learning

Notes for Assignment:

1. **Most important: working code, reasonable plot, and analysis**
2. **Make your answer to each question self-contained and unambiguous using scientific language**
3. **Use clear idiomatic code, variable, and function names that make code as self-explanatory as possible (add comments where appropriate)**

Module 3:

Linear Models for Classification

(Part A)

Module Objectives

Understand

- Basic Concepts of Classification
- Discriminative Models / Geometric Perspective
- Extension to Multiple Classes
- Historic Note: Perceptron Algorithm
- Probabilistic Model: Logistic Regression
- Generative Models: Bayes Classifier (Single Input)
- Handling Multiple Inputs: Naïve Bayes and Multivariate Gaussian

Basic Concepts of
Linear Models for Classification

Regression vs. Classification

Regression: model relationship between input variables and **numerical target variable** (typically continuous or well approximated by continuous variable).

- predicting house prices based on location, size, and age of houses
- predicting exam scores based on hours of different modes of study
- predicting movie ticket sales based on number of tweets

Classification: model relationship between input variables and **discrete target variable** with K possible outcomes that do no have a clear order.

- Predict whether an email is *spam/no-spam* to deliver to junk folder ($K = 2$)
- Classify customers into good credit, bad credit, or grey ($K = 3$)
- Recognise hand-written characters ($K = 34$ or more)

Intuition: Linear Classification Model

An example of training data

Instance	x_1 (Length)	x_2 (Width)	Class (T)
1	100	50	TUNA
2	60	20	BASS
3	90	20	TUNA
4	50	30	BASS
…	…	…	…

Simple Linear Classifier

Learn weights:

$$a(\boldsymbol{x}) = 3x_1 + 2x_2 - 250$$

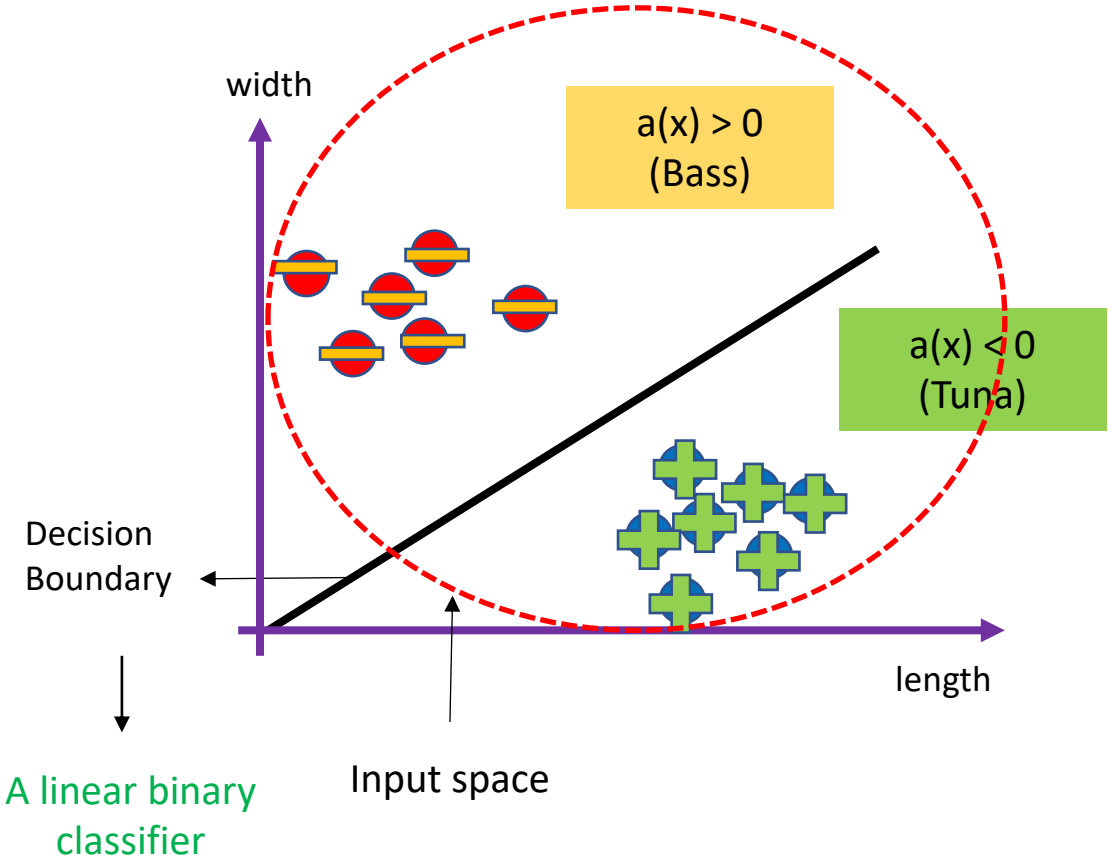
Feature	Weights
1	250
x_1	3
x_2	2

Prediction function:

$$y(\boldsymbol{x}, \boldsymbol{w}) = \begin{cases} \text{TUNA} & \text{, if } a(\boldsymbol{x}) > 0 \\ \text{BASS} & \text{, otherwise} \end{cases}$$

Geometric View

Suppose only two input variables had non-zero weights



- Input Space: each input data point is represented by a vector in the input space
- Decision Regions: input space is divided into decision regions, each corresponding to a predicted class
- Decision Boundary: the decision regions are separated by decision boundaries
- Linear Models: decision boundaries are linear equations of input vector:
  - for d-dimensional inputs, decision boundary is a $(d-1)$-dimensional "hyperplane".
- Linear Separability: datasets whose classes can be separated exactly by linear decision boundary are called linearly separable

Formal Classification problem

Given

- data $D = (\boldsymbol{x}_1, t_1), \dots, (\boldsymbol{x}_N, t_N)$
- sampled according to joint distribution of RVs X and T
- input variable(s) X often assumed with $D_X = \mathbb{R}^d$
- discrete target variable T with finite domain $D_T = \{c_1, \dots, c_K\}$
- elements are called classes or labels (e.g. “spam”, “non-spam”; or “good”, “bad”, “grey”)

Goal

- fit classification model that predicts/describes well class T given input $X = \boldsymbol{x}$

Three Flavours of Classification Models

Predictive or *Discriminative* Models

- Provides prediction $y(\boldsymbol{x}, \boldsymbol{w})$ of class c_k associated with input $\boldsymbol{x}$
- Equivalent to what we just called "predictive model" in regression

Probabilistic (Discriminative) Models

- Provide probabilities for each class
- Defined by conditional distribution for class given input (and parameters) $p(c_k|\boldsymbol{x}, \boldsymbol{w})$
- Naturally also provides prediction $y(\boldsymbol{x}, \boldsymbol{w}) = \text{argmax}\{p(c_k|\boldsymbol{x}, \boldsymbol{w}): c_k \in D_T\}$

Generative Models

- Models full joint distribution of X and T
- Usually via class conditional density of input given class and class prior

$$p(c_k|\boldsymbol{x}) \propto p(c_k)p(\boldsymbol{x}|c_k)$$

- Can be used to *generate* input variables

Formal Classification problem

Given

- data $D = (\boldsymbol{x}_1, t_1), \dots, (\boldsymbol{x}_N, t_N)$
- sampled according to joint distribution of RVs X and T
- input variable(s) X often assumed with $D_X = \mathbb{R}^d$
- discrete target variable T with finite domain $D_T = \{c_1, \dots, c_K\}$
- elements are called classes or labels (e.g. “spam”, “non-spam”; or “good”, “bad”, “grey”)

Goal

- learn parameters $\boldsymbol{w}$ of prediction function $y(\boldsymbol{x}, \boldsymbol{w})$ [or probabilistic model $p(c_k|\boldsymbol{x}, \boldsymbol{w})$]
- that minimise generalisation error (prediction risk)

$$\mathbf{E}\big[l\big(T, y(X, \boldsymbol{w})\big)\big]$$

- …with some loss function l such as 0/1-loss

$$l(t, y) = \begin{cases} 0, & \text{if } t = y \\ 1, & \text{otherwise} \end{cases}$$

Discriminative Models for Binary Classification

Binary Classification Problems

We focus on *binary* classification first

- only two classes $D_T = \{c_1, c_2\}$
- typically identified with numbers
  - 0 (called negative class)
  - 1 (called positive class)
- sometimes also identify negative class with -1

Generalised Linear Models

We can reuse linear regression models for classification

Linear (Regression) Model: Prediction $y(\boldsymbol{x})$ given by linear function of weight parameters $\boldsymbol{w} \in \mathbb{R}^{\boldsymbol{d}}$ and intercept parameter w_0:

$$y(\boldsymbol{x}) = w_0 + \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$$

also called "offset", "threshold", or "bias"

Generalised Linear Model (e.g., for Classification): linear discriminant function $a(\boldsymbol{x}) = w_0 + \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$ input to non-linear activation function f:

$$y(\boldsymbol{x}) = f\big(a(\boldsymbol{x})\big) = f(w_0 + \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})$$

can still include non-linear feature transformation ϕ but we won't denote it here

Examples:

- step function (Perceptron, discriminative):

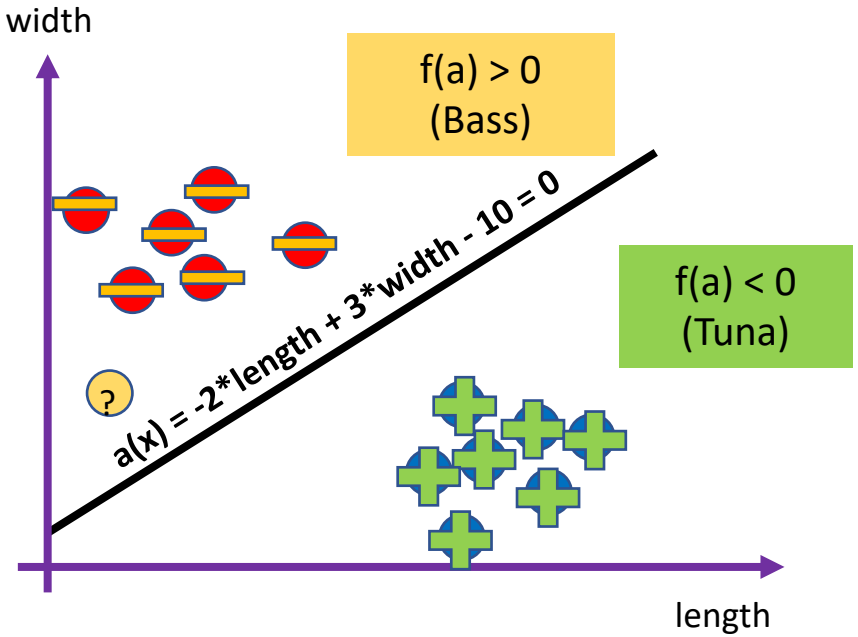
$$f(a) = \begin{cases} 1 & \text{if } a \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- logistic function (Logistic regression, probabilistic):

$$f(a) = 1/(1 + \exp(-a))$$

Linear Classifier

Decision regions with linear discriminant function and step activation

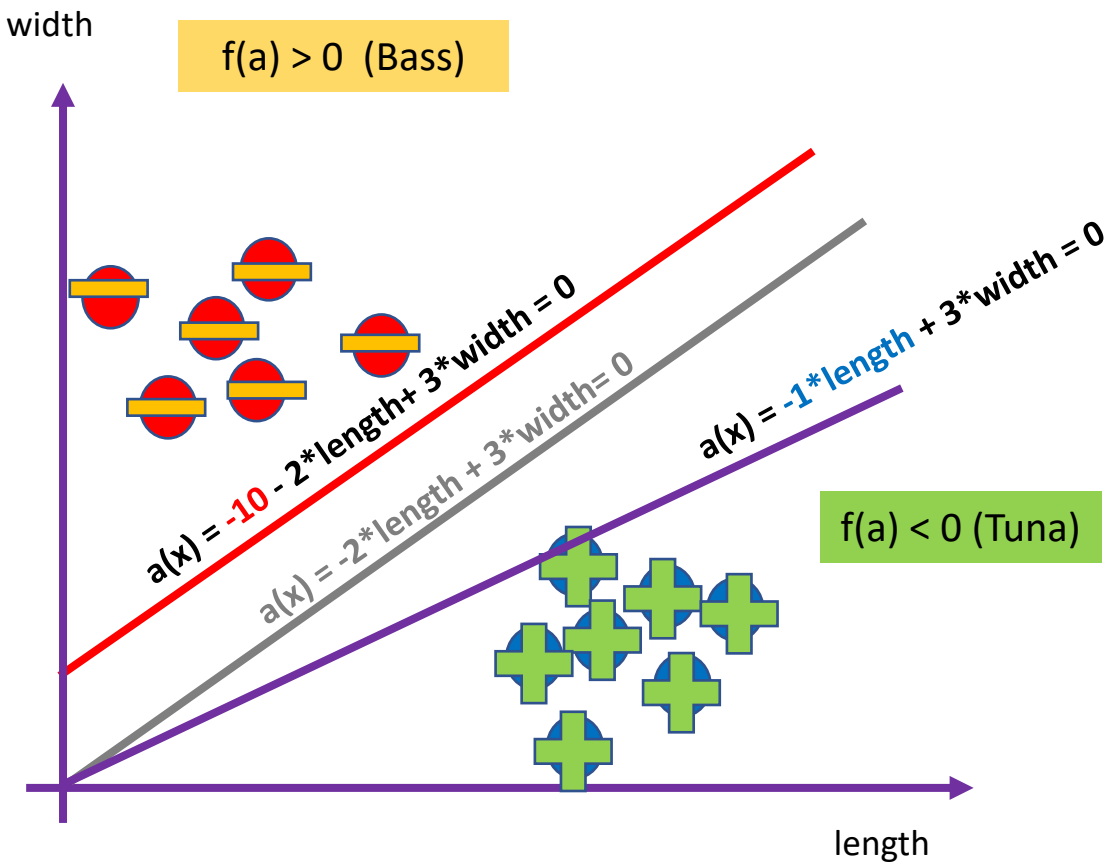


$$y(\boldsymbol{x}) = f\big(a(\boldsymbol{x})\big) = \begin{cases} 1 \text{ (Tuna)} & \text{if } a(\boldsymbol{x}) \geq 0 \\ 0 \text{ (Bass)} & \text{otherwise} \end{cases}$$

Parameters:

Input	Parameter	Value
intercept	w_0	-10
Length weight	w_1	-2
width weight	w_2	3

Linear Classifier: Effect of Parameters



Change Input Weight:

Input	Parameter	Value
length	w_1	-2 => -1
width	w_2	3

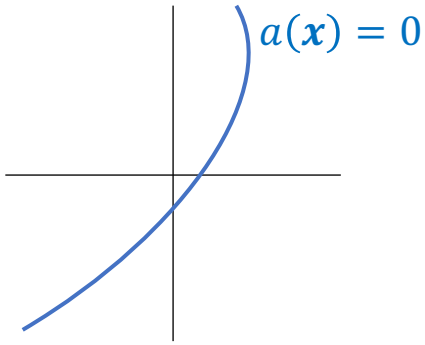
Add intercept:

Input	Parameter	Value
intercept	w_0	-10
length	w_1	-2

Decision Boundary

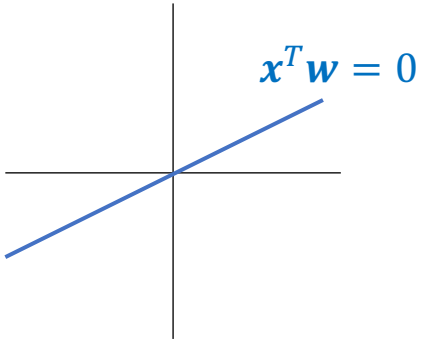
Using step function $f(a) = \delta(a > 0)$ results in decision boundary where discriminant function $a(x)$ is 0:

$$B = \{x \in \mathbb{R}^d : a(\boldsymbol{x}) = 0\}$$



For linear discriminant function $a(\boldsymbol{x}) = \boldsymbol{x}^T\boldsymbol{w}$ (without offset) results in linear subspace, meaning $\mathbf{0} \in B$ (i.e., passes origin) and $\alpha\boldsymbol{x} + \beta\boldsymbol{x}' \in B$ if $\boldsymbol{x}, \boldsymbol{x}' \in B$:

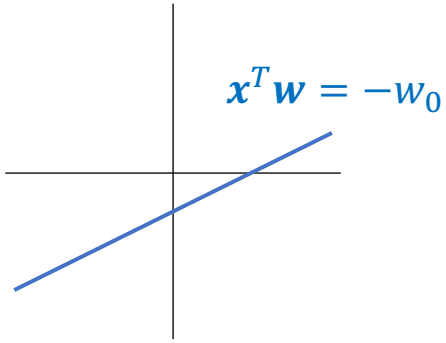
$$B = \{x \in \mathbb{R}^d : \boldsymbol{x}^T\boldsymbol{w} = 0\}$$



…of dimension $d - 1$, because fixing $x_1, \dots, x_{d-1}$ determines $x_d = -\sum_{i=1}^{d-1} x_i w_i$.

When including offset $a(\boldsymbol{x}) = \boldsymbol{x}^T\boldsymbol{w} + w_0$ boundary is shifted (affine subspace)

$$B = \{x \in \mathbb{R}^d : \boldsymbol{x}^T\boldsymbol{w} = -w_0\}$$



This is called hyperplane, e.g., a line if $d = 2$, a plane if $d = 3$.

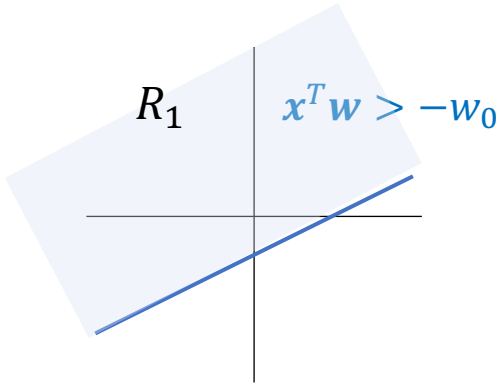
Decision Regions

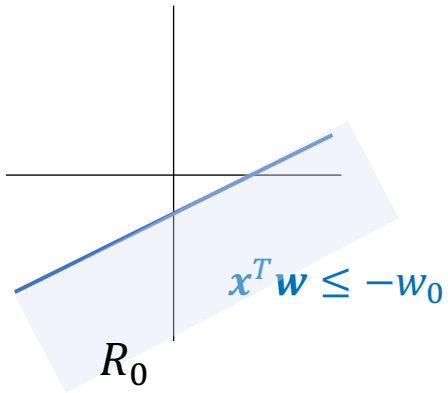
Decision regions are given by inequalities

$$R_1 = \{x \in \mathbb{R}^2 : a(\boldsymbol{x}) > 0\}$$
$$R_0 = \{x \in \mathbb{R}^2 : a(\boldsymbol{x}) \leq 0\}$$

Choice on boundary is arbitrary, with step function as before, they are part of negative region

For linear discriminant, regions are half-spaces: subsets of input space described by linear inequality



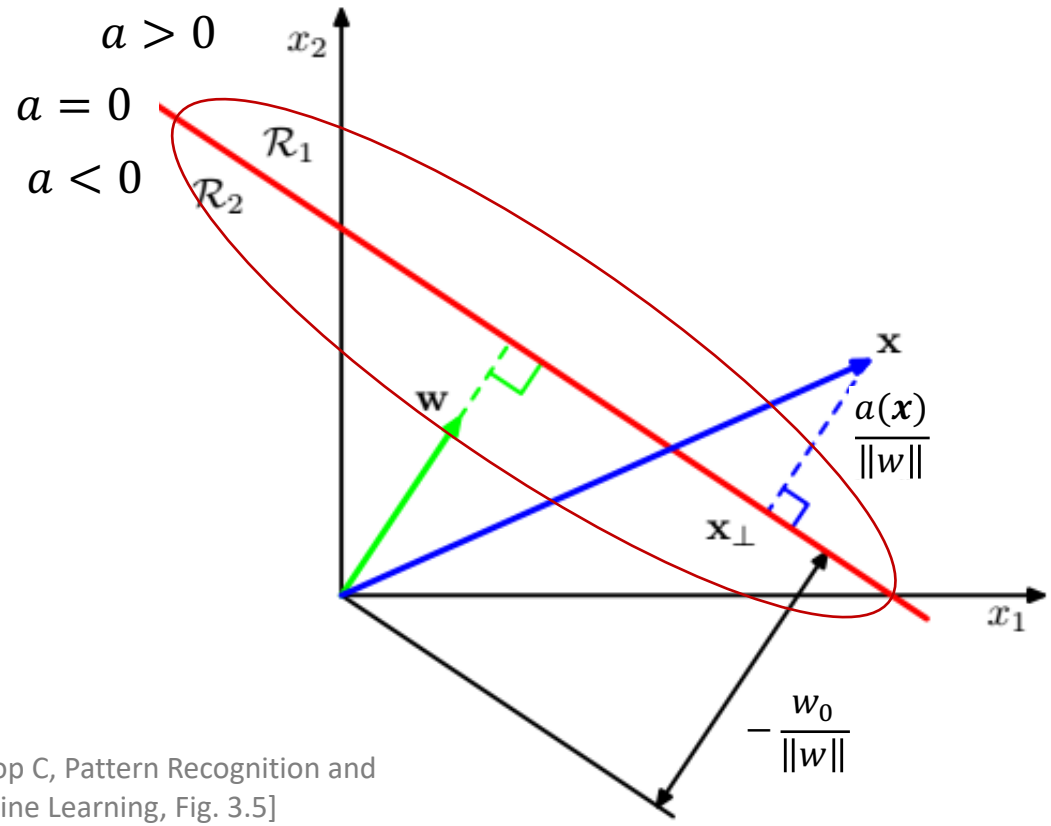


Relation of Decision Boundary and Weight Vector

Decision Boundary

$$\boldsymbol{w}^T \boldsymbol{x} + w_0 = 0$$

Geometry



[Bishop C, Pattern Recognition and Machine Learning, Fig. 3.5]

Assume inputs $\boldsymbol{x}_1$ and $\boldsymbol{x}_2$ on same "contour" of a, i.e.,

$$a(\boldsymbol{x}_1) = a(\boldsymbol{x}_2)$$

$$\Updownarrow$$

$$\boldsymbol{w} \cdot \boldsymbol{x}_1 + w_0 = \boldsymbol{w} \cdot \boldsymbol{x}_2 + w_0$$

$$\Updownarrow$$

$$\boldsymbol{w} \cdot (\boldsymbol{x}_1 - \boldsymbol{x}_2) = 0$$

Thus: decision boundary is orthogonal to weight vector

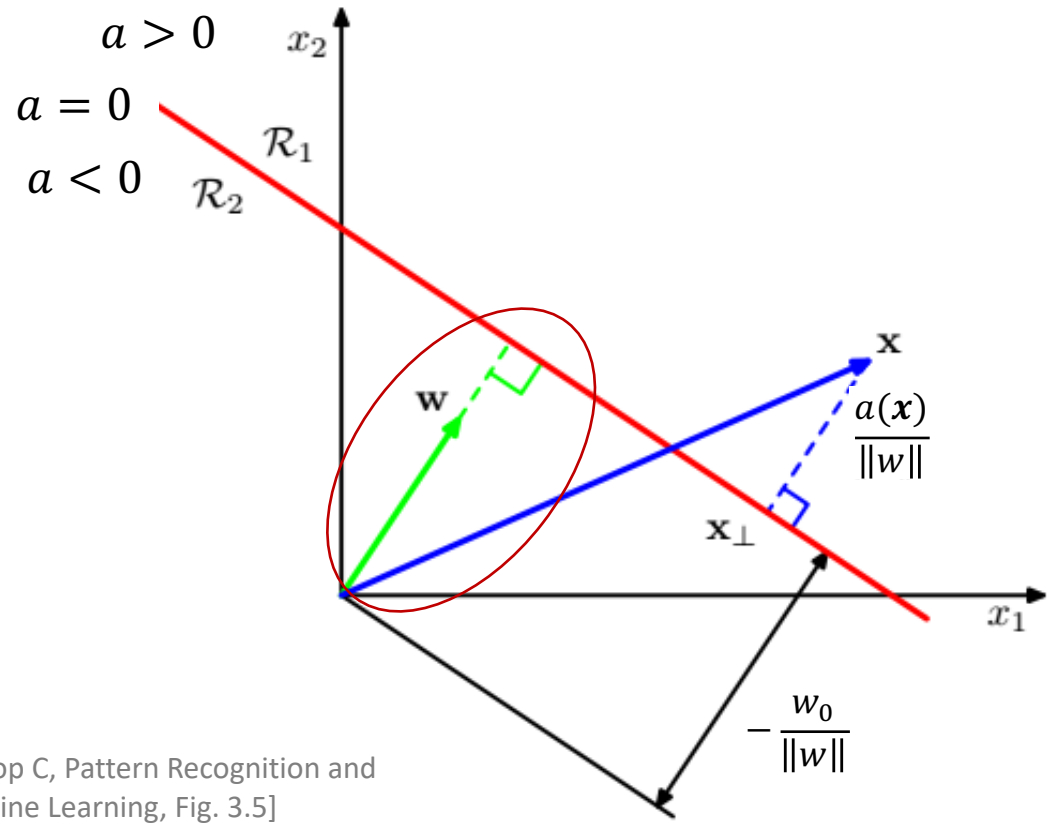
(case $a(\boldsymbol{x}_1) = a(\boldsymbol{x}_2) = 0$)

Relation of Decision Boundary and Intercept

Decision Boundary

$$\boldsymbol{w}^T \boldsymbol{x} + w_0 = 0$$

Geometry



[Bishop C, Pattern Recognition and Machine Learning, Fig. 3.5]

Consider projection of point $\boldsymbol{x}$ from decision boundary onto $\boldsymbol{w}$:

$$a(\boldsymbol{x}) = \boldsymbol{w} \cdot \boldsymbol{x} + w_0 = 0$$

$$\Updownarrow$$

$$\boldsymbol{w} \cdot \boldsymbol{x} = -w_0$$

$$\Downarrow$$

$$\frac{\boldsymbol{w} \cdot \boldsymbol{x}}{\boldsymbol{w} \cdot \boldsymbol{w}} \boldsymbol{w} = \frac{\boldsymbol{w} \cdot \boldsymbol{x}}{\|\boldsymbol{w}\|} \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|} = \frac{(-w_0)}{\|\boldsymbol{w}\|} \frac{\boldsymbol{w}}{\|\boldsymbol{w}\|}$$

$\Longrightarrow$ distance of boundary from origin

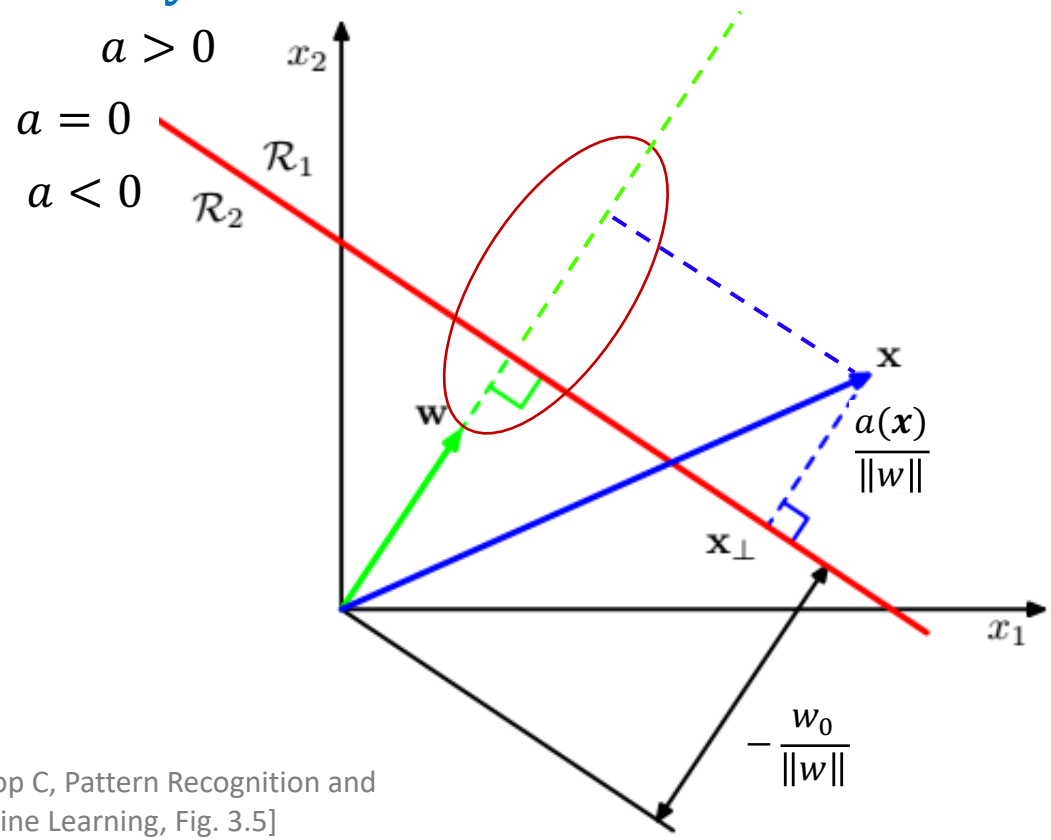
$$-\frac{w_0}{\|\boldsymbol{w}\|}$$

Distance of Inputs from Decision Boundary

Decision Boundary

$$\boldsymbol{w}^T\boldsymbol{x} + w_0 = 0$$

Geometry



[Bishop C, Pattern Recognition and Machine Learning, Fig. 3.5]

Consider length of projection of $\boldsymbol{x}$ onto weight vector:

$$\left\|\frac{\boldsymbol{w}\cdot\boldsymbol{x}}{\|\boldsymbol{w}\|}\frac{\boldsymbol{w}}{\|\boldsymbol{w}\|}\right\| = \frac{\boldsymbol{w}\cdot\boldsymbol{x}}{\|\boldsymbol{w}\|}$$

$$= \frac{a(\boldsymbol{x}) - w_0}{\|\boldsymbol{w}\|}$$

and subtract distance of boundary from origin $(-w_0/\|\boldsymbol{w}\|)$

$\Longrightarrow$ distance of $\boldsymbol{x}$ from boundary

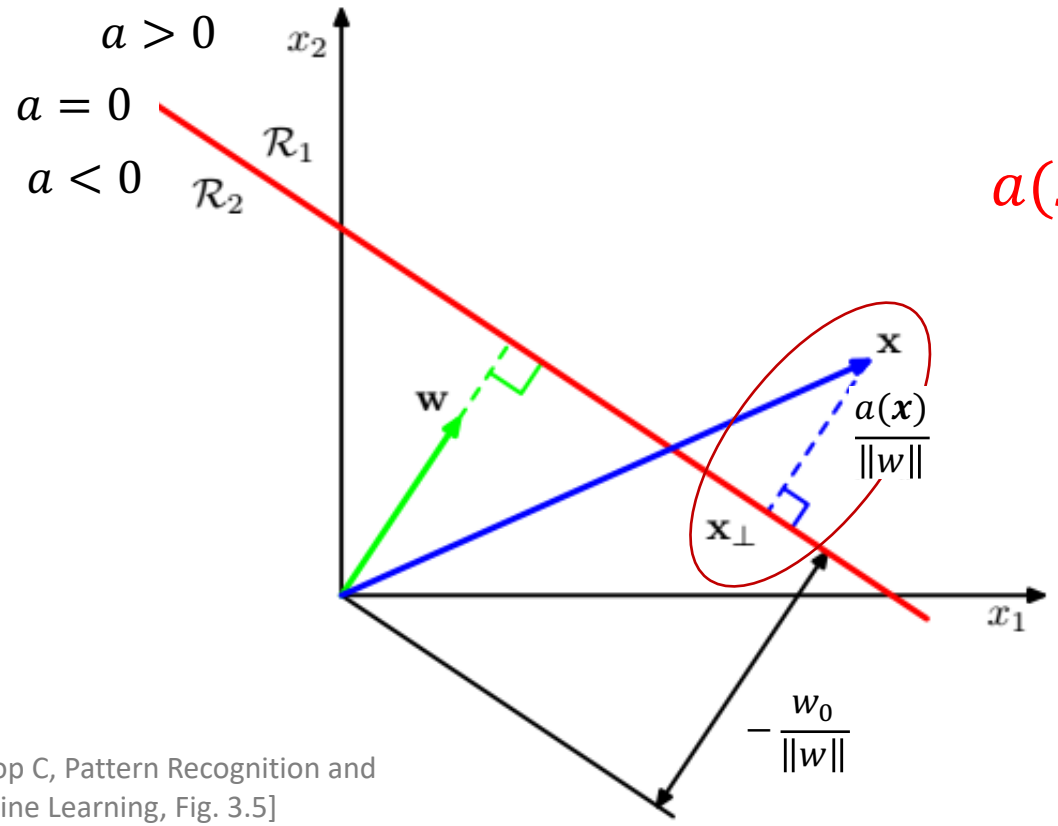
$$\frac{a(\boldsymbol{x})}{\|\boldsymbol{w}\|}$$

Alternative Argument from Bishop, p. 182

Decision Boundary

$$\boldsymbol{w}^T\boldsymbol{x} + w_0 = 0$$

Geometry



[Bishop C, Pattern Recognition and Machine Learning, Fig. 3.5]

Represent $\boldsymbol{x}$ via its projection $\boldsymbol{x}_\perp$ onto boundary:

$$\boldsymbol{x} = \boldsymbol{x}_\perp + r\frac{\boldsymbol{w}}{\|\boldsymbol{w}\|}$$

with

$$a(\boldsymbol{x}_\perp) = \boldsymbol{w}\cdot\boldsymbol{x}_\perp + w_0 = 0$$

Then

$$\boldsymbol{w}\cdot\boldsymbol{x} = \boldsymbol{w}\cdot\boldsymbol{x}_\perp + r\frac{\boldsymbol{w}\cdot\boldsymbol{w}}{\|\boldsymbol{w}\|}$$

$a(\boldsymbol{x})$

$$\boxed{\boldsymbol{w}\cdot\boldsymbol{x} + w_0} = \boxed{\boldsymbol{w}\cdot\boldsymbol{x}_\perp + w_0} + r\|\boldsymbol{w}\|$$

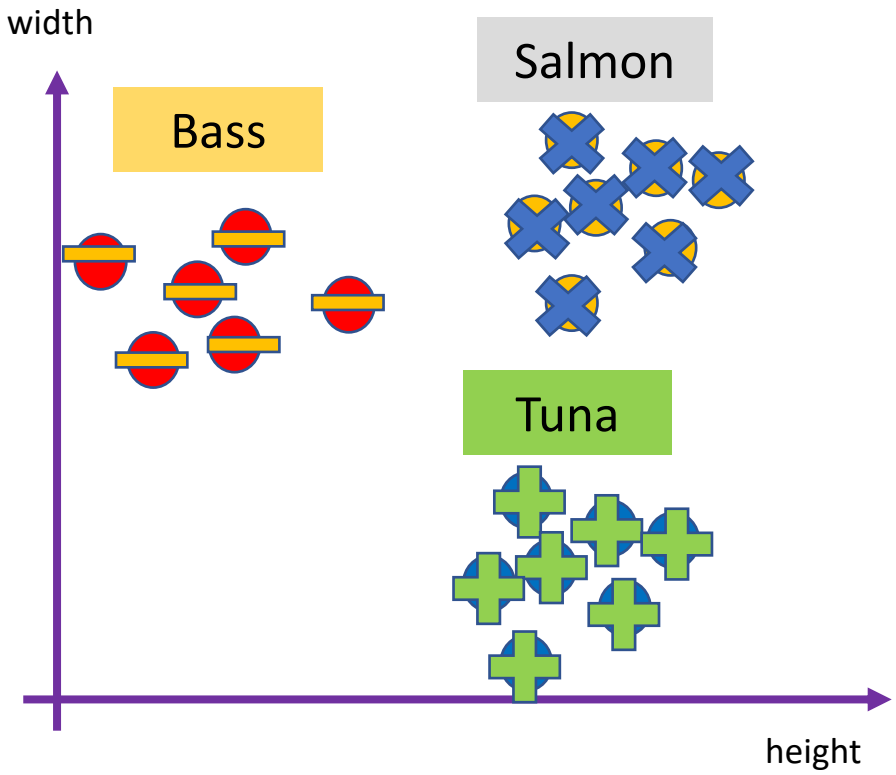
$\Longrightarrow$ distance of $\boldsymbol{x}$ from boundary

$$r = \frac{a(\boldsymbol{x})}{\|\boldsymbol{w}\|}$$

Discriminative Models for Multiclass Classification

Generalisation to Multiclass Problems

How to to define classifiers when there are $K > 2$ classes?

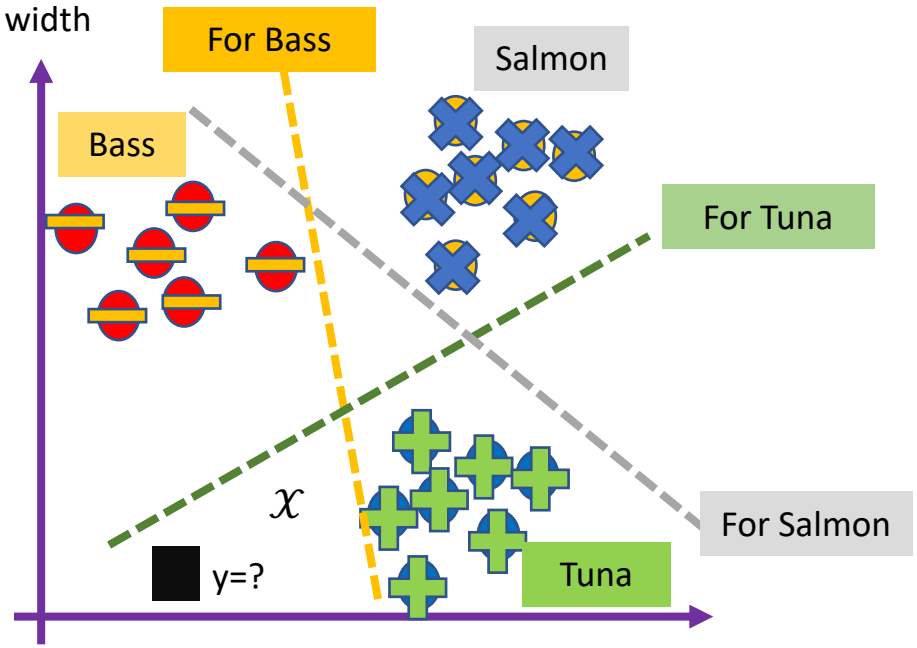


Generalisation to Multiclass Problems

One-versus-the-rest classifier

Idea:

- Transfer multi-class classification into binary classification problem.
- Use K discriminant functions, each separating one class c_k from the rest
- Problem: Ambiguous regions: $\mathcal{X} \subseteq D_X$

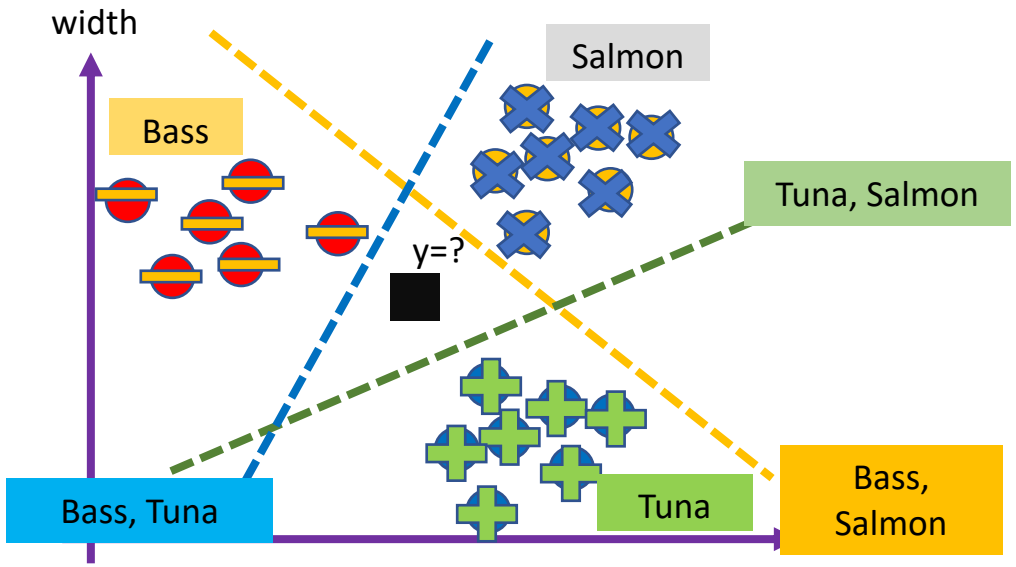


Generalisation to Multiclass Problems

One-versus-one classifier

Idea:

- Build binary classifier for each pair of classes
- Use $k(k-1)/2$ discriminant functions, each for every possible pair of classes
- Each point is classified by majority vote
- Problem: Ambiguous regions: $\mathcal{X} \subseteq D_X$



Generalisation to Multiclass Problems

Use K discriminant functions $a_k(\mathbf{x})$

Idea:

- Use the magnitude of $a_k(\mathbf{x})$, not just the sign
- For each class k, we consider a linear function of the form: $a_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathrm{T}}\boldsymbol{x} + w_{k0}$
- We then assign a point $\boldsymbol{x}$ to a class c_k if $a_k(\mathbf{x}) > a_j(\mathbf{x})$ for all $j \neq k$
- That is, $y(\boldsymbol{x}) = \underset{k}{\operatorname{argmax}}\, a_k(\mathbf{x})$

Alternative formalisation:

- Consider vector-valued activation function $f(\boldsymbol{a}) = (f_1(\boldsymbol{a}), \dots, f_K(\boldsymbol{a}))$, e.g.,

$$f_i(\boldsymbol{a}) = \begin{cases} 1, & \text{if } a_i = \max\{a_1, \dots, a_K\} \\ 0, & \text{otherwise.} \end{cases}$$

- For probabilistic model can then use soft-max

$$f_i(\boldsymbol{a}) = \frac{\exp a_k}{\sum_{k=1}^{K} \exp a_k} = p(T = c_i \mid X = \boldsymbol{x})$$

Generalisation to Multiclass Problems

Decision Regions

- Intersection of half-spaces $R_k = \{x \in \mathbb{R}^d : a_k(x) \geq a_i(x) \text{ for all } i \neq k\}$
- No ambiguous region (except boundary)
- Connected and convex: for $\boldsymbol{x}_A, \boldsymbol{x}_B \in R_k$ also $\hat{\boldsymbol{x}} = \alpha \boldsymbol{x}_A + (1-\alpha)\boldsymbol{x}_B \in R_k$ for $\alpha \in [0,1]$

Decision Boundary

- Boundary between class i and j:

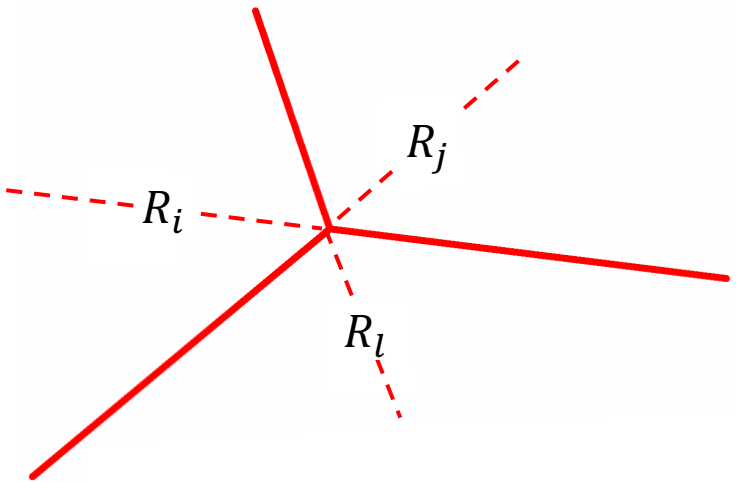
$$B_{ij} = \left\{\boldsymbol{x} \in \mathbb{R}^2 : (\boldsymbol{w_i} - \boldsymbol{w_j})^{\boldsymbol{T}} \boldsymbol{x} = w_{j0} - w_{i0}\right\}$$

- Relevant part where it intersects corresponding regions

$$B'_{ij} = B_{ij} \cap R_i \cap R_j$$

- Overall decision boundary union of relevant parts

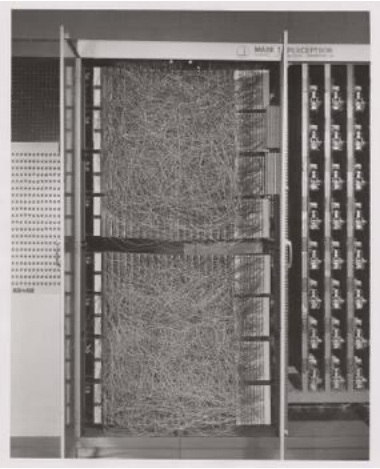
$$B = B'_{1,2} \cup B'_{1,3} \cup \cdots \cup B'_{K-1,K}$$

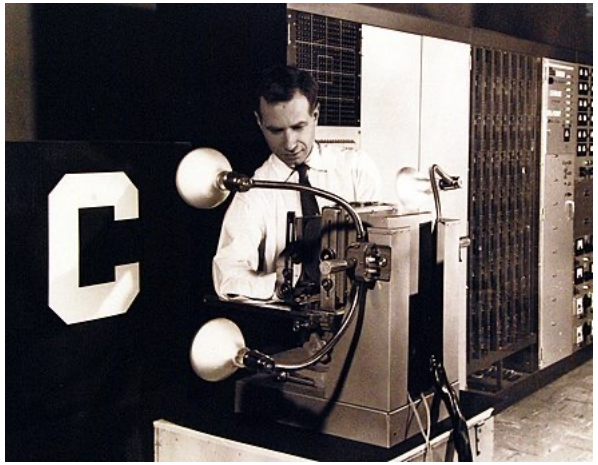


The Perceptron Algorithm

Historical Note

- Perceptron algorithm fits the weights of a linear discriminative model
- Perhaps the earliest AI algorithm/system: *McCulloch, W; Pitts, W (1943). "A Logical Calculus of Ideas Immanent in Nervous Activity". Bulletin of Mathematical Biophysics. 5 (4)*





Images: Wikipedia

- Not relevant today as an algorithm (replaced by stochastic gradient descent), but still nice didactical introduction to Artificial Neural Networks

Perceptron

Two-class Perceptron

- The simplest form of a neural network that can be used as a linear classifier
- A generalised linear model in which the input vector $\boldsymbol{x}$ is transformed non-linearly:

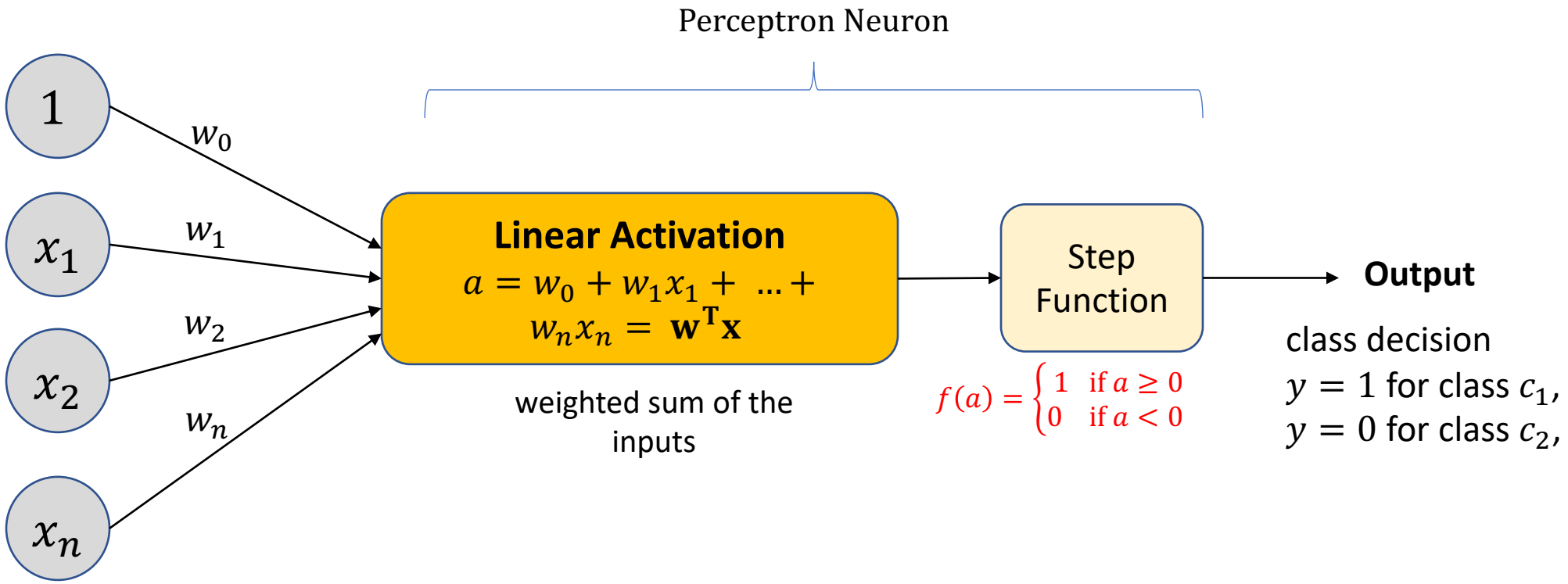
$$y(\mathbf{x}) = f(\boldsymbol{w} \cdot \mathbf{x})$$

where the nonlinear activation function $f(\cdot)$ is given by a step function:

$$f(a) = \begin{cases} 1 & \text{if } a \geq 0 \\ 0 & \text{if } a < 0 \end{cases}$$

- The parameters $\boldsymbol{w}$ are sometimes called weights (real-valued constants)

Perceptron Classifier



- The neuron receives multiple inputs and process them to produce an output
- The perceptron calculates 2 quantities: (1) weighted sum of the input features and (2) this sum is then thresholded by the *f* function.

Perceptron Model

Perceptron Model

- Input: $\mathbf{x}$
- Parameter: $\mathbf{w}$
- Decision Boundary: $\mathbf{w}^{\mathrm{T}}\mathbf{x} = 0$ (for simplicity, we assume $\phi(\mathbf{x}) = (1, \mathbf{x})$)
- Classifier: $y(\mathbf{x}) = \begin{cases} +1 & \text{if } \mathbf{w}^{\mathrm{T}}\mathbf{x} \geq 0 \\ -1 & \text{Otherwise} \end{cases}$

Perceptron Learning

Perceptron Learning

- Goal: find parameters $\mathbf{w}$ that minimises number of misclassified training data points.
- Can again be formalised as minimising an error function.

Error Function

- **Intuition:** minimise magnitude of $a(\boldsymbol{x})$-value of misclassified training points $\boldsymbol{x}$.
- Magnitude of misclassification: $e(\boldsymbol{w}) = \begin{cases} \boldsymbol{w}^T\boldsymbol{x} & \text{if } t = 0 \\ -\boldsymbol{w}^T\boldsymbol{x} & \text{if } t = 1 \end{cases} = -\boldsymbol{w}^T\boldsymbol{x}(2t-1)$
- Definition of an error function of Perceptron:

$$E(\mathbf{w}) = -\sum_{n \in M} \mathbf{w} \cdot \mathbf{x}_n (2t_n - 1)$$

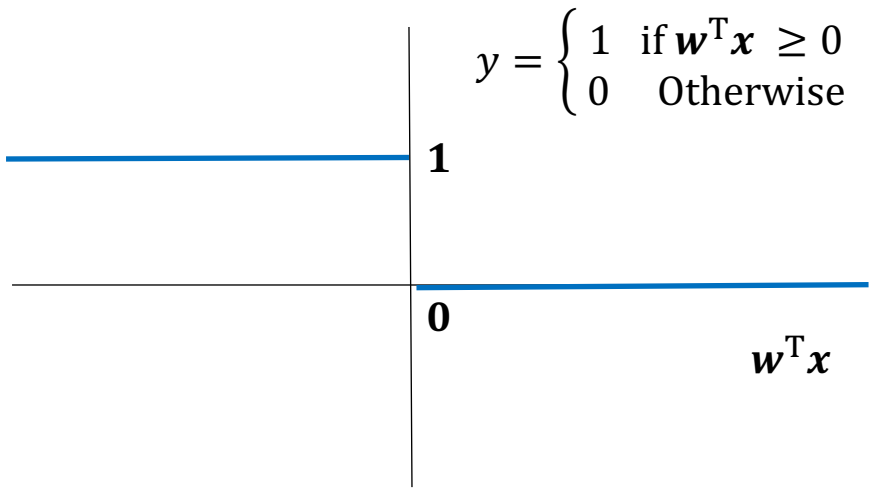
where M is set of *misclassified* examples, and t_n is the true label of $\boldsymbol{x}_n$

Perceptron Learning Algorithm

View as training risk minimisation

Can rewrite losses as function of a

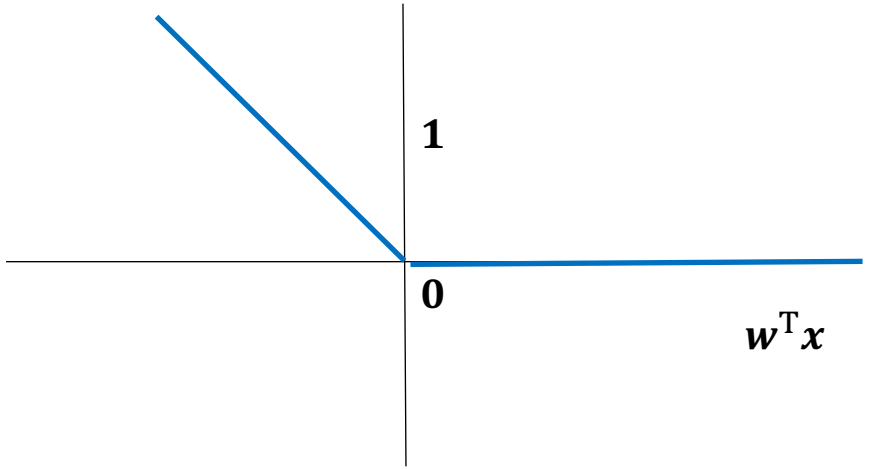
$$l_{0/1}(t, a) = \begin{cases} 0, & \text{if } t = \delta(a \geq 0) \\ 1, & \text{otherwise.} \end{cases}$$



$t = 1$

Perceptron loss

$$l_{\text{perc}}(t, a) = \begin{cases} 0, & \text{if } t = \delta(a \geq 0) \\ |a|, & \text{otherwise.} \end{cases}$$



Perceptron Learning Algorithm

Initialise $\mathbf{w}$ with random values

Repeat until all data points are classified correctly or the error $< \epsilon$:

- For each training point $\mathbf{x}$, perform the following
  - Compute the activation function

$$y = \begin{cases} 1 & \text{if } \mathbf{w}^{\mathrm{T}}\mathbf{x} \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

  - If $y = t$, don't change the weights
  - Else (i.e. $y \neq t$) , update the weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(2t - 1)\boldsymbol{x}_n$$

(η: learning rate)

Form of stochastic gradient descent
(let M be indices of misclassified ex.)

$$E(\mathbf{w}) = -\sum_{n \in M} \mathbf{w} \cdot \mathbf{x}_n (2t_n - 1)$$

$$\nabla E(\mathbf{w}) = -\sum_{n \in M} \mathbf{x}_n (2t_n - 1)$$

Note: As the parameters change, the data points that were correctly modified might change to being misclassified

Perceptron Learning Example

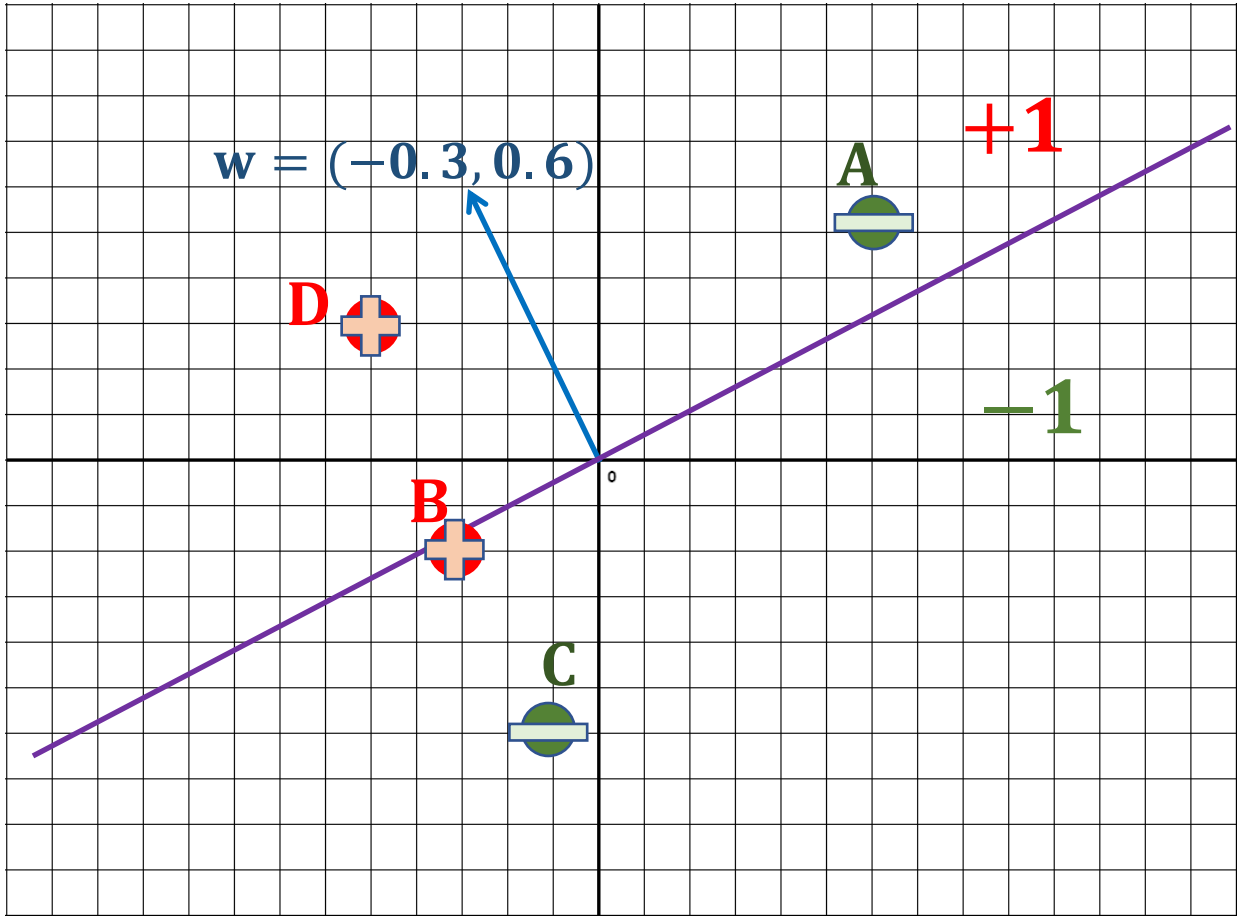
Training data

Item	x_1	x_2	Class
A	0.6	0.5	0
B	-0.3	-0.2	1
C	-0.1	-0.6	0
D	-0.5	0.3	1

For input A

- $\mathbf{w}^{\mathrm{T}}\mathbf{x} = -0.3 \cdot 0.6 + 0.6 \cdot 0.5 = 0.12 \ \geq 0$
- $y = 1$ but $t = 0$
- Update the weights
  - direction $(2t - 1) = -1$
  - $w_1 = w_1 - x_1 = -0.3 - 0.6 = -0.9$
  - $w_2 = w_2 - x_2 = 0.6 - 0.5 = 0.1$

Learning rate: $\eta = 1$, initial $\mathbf{w} = (-0.3, 0.6)$



Perceptron Learning Example

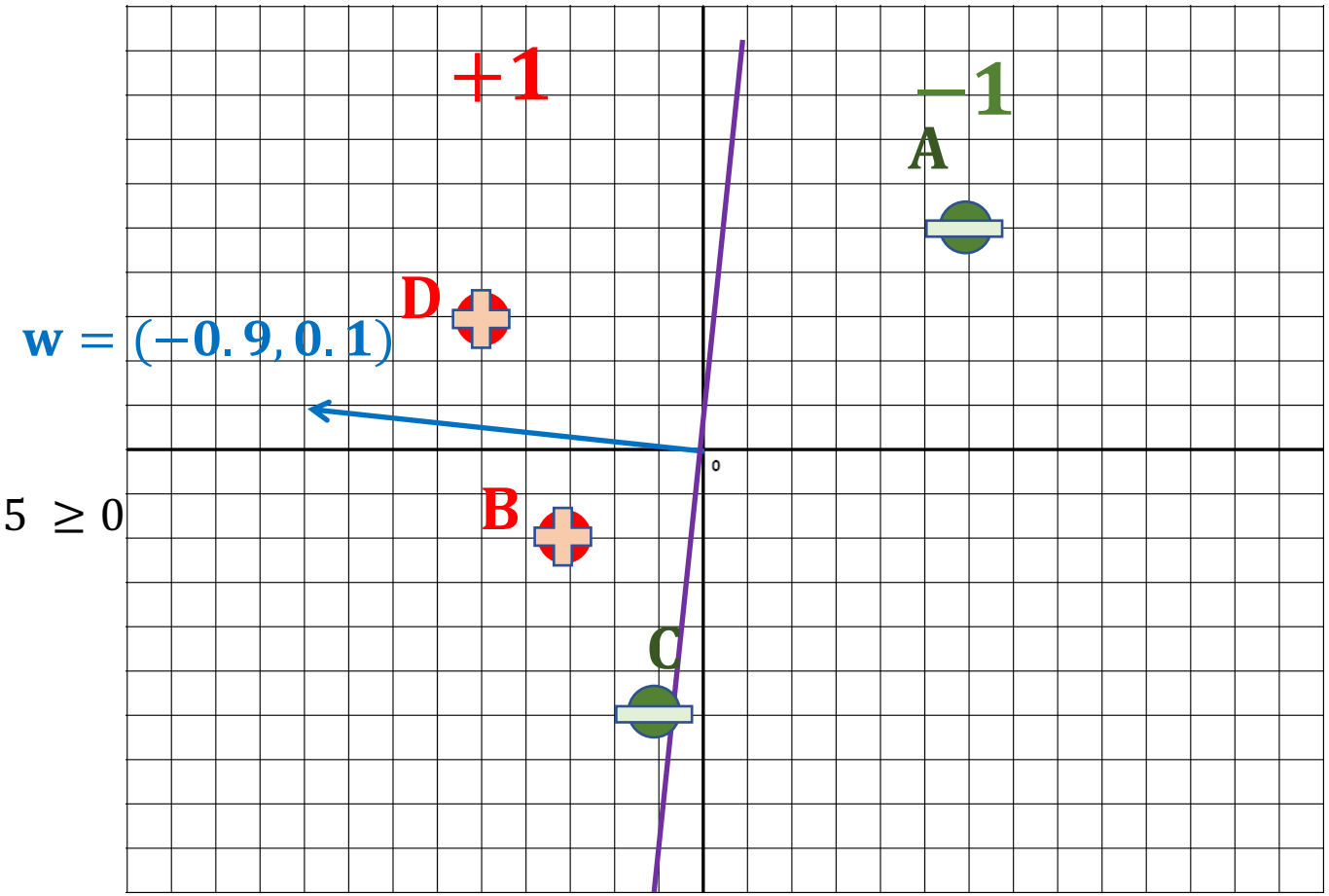
Training data

Item	x_1	x_2	Class
A	0.6	0.5	0
B	-0.3	-0.2	1
C	-0.1	-0.6	0
D	-0.5	0.3	1

For input B

- $\mathbf{w}^{\mathrm{T}}\mathbf{x} = -0.9 \cdot (-0.3) + 0.1 \cdot (-0.2) = 0.25 \ \geq 0$
- $y = 1$, and $t = 1$
- Don't update the weights

Learning rate: $\eta = 1$, $\mathbf{w} = (-0.9, 0.1)$

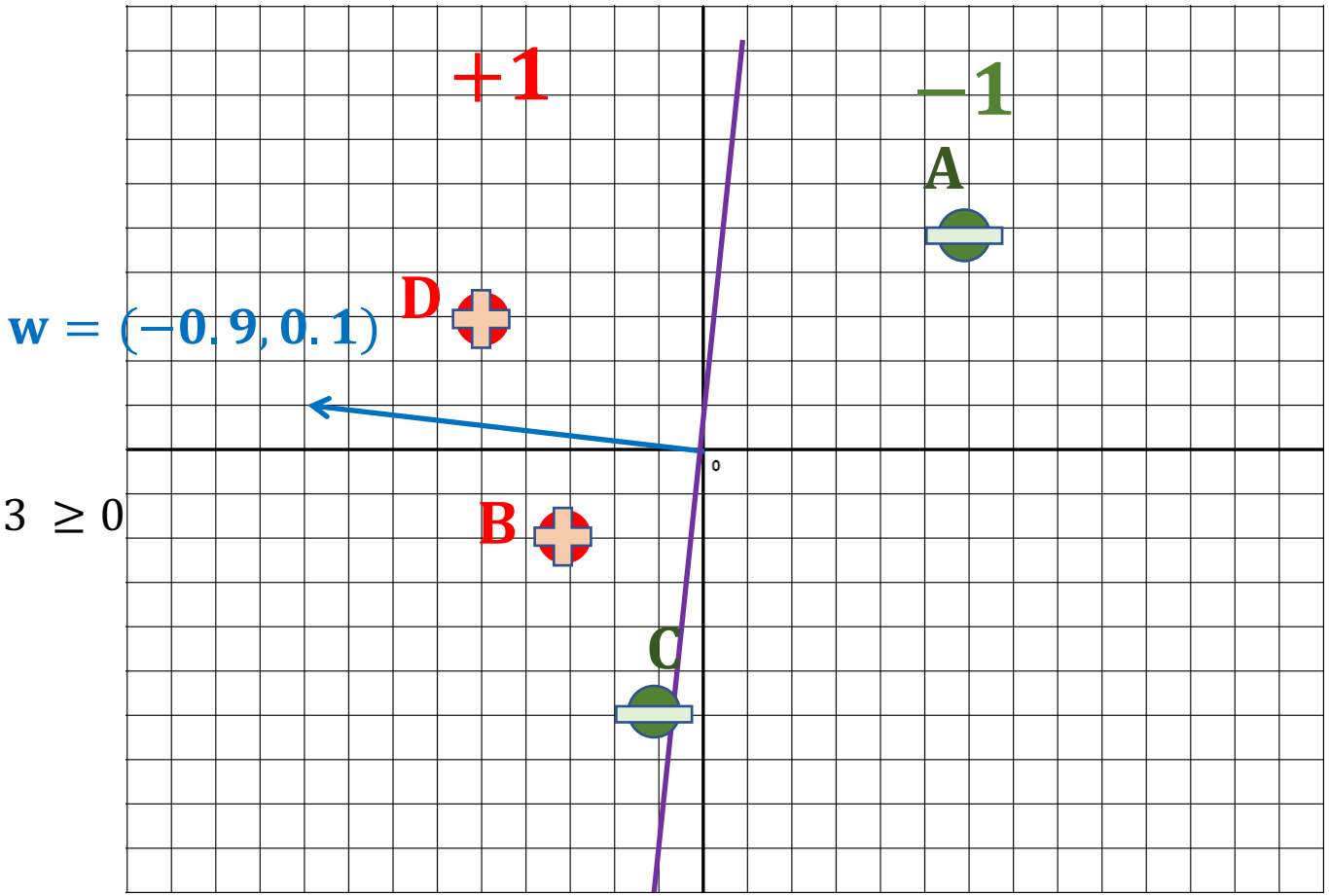


Perceptron Learning Example

Training data

Item	x_1	x_2	Class
A	0.6	0.5	0
B	-0.3	-0.2	1
C	-0.1	-0.6	0
D	-0.5	0.3	1

Learning rate: $\eta = 1$, $\mathbf{w} = (-0.9, 0.1)$



For input C

- $\mathbf{w}^{\mathrm{T}}\mathbf{x} = -0.9 \cdot (-0.1) + 0.1 \cdot (-0.6) = 0.03 \ \geq 0$
- $y = 1$, and $t = 0$
- Update the weights
  - direction $(2t - 1) = -1$
  - $w_1 = w_1 - x_1 = -0.9 + 0.1 = -0.8$
  - $w_2 = w_2 - x_2 = 0.1 + 0.6 = 0.7$

Perceptron Learning Example

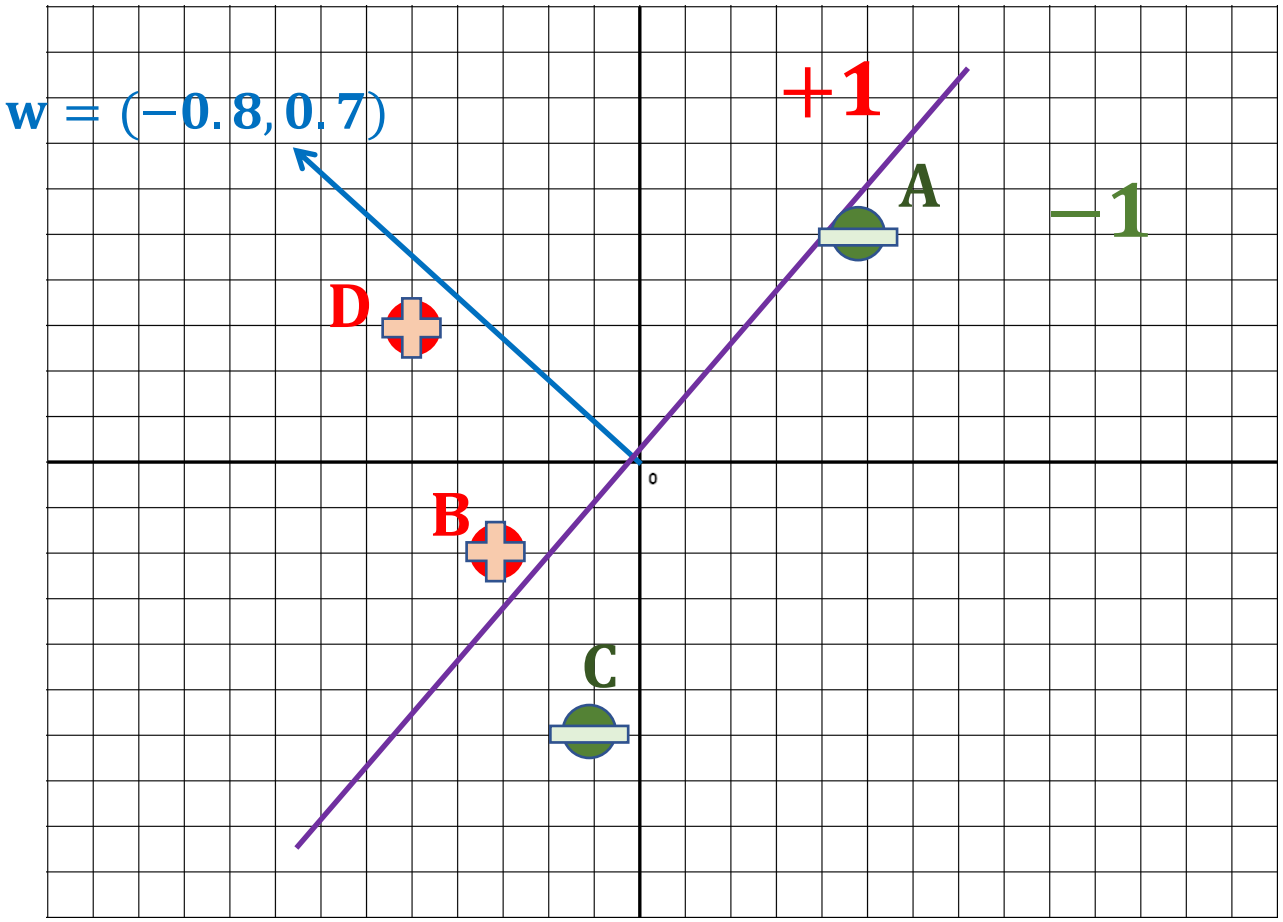
Training data

Item	x_1	x_2	Class
A	0.6	0.5	0
B	-0.3	-0.2	1
C	-0.1	-0.6	0
D	**-0.5**	**0.3**	**1**

For input D

- $\mathbf{w}^{\mathrm{T}}\mathbf{x} = -0.8 \cdot (-0.5) + 0.7 \cdot 0.3 = 0.61 \ \geq 0$
- $y = 1$, and $t = 1$
- Don't update the weights

Learning rate: $\eta = 1$, $\mathbf{w} = (-0.8, 0.7)$



Perceptron Learning Example

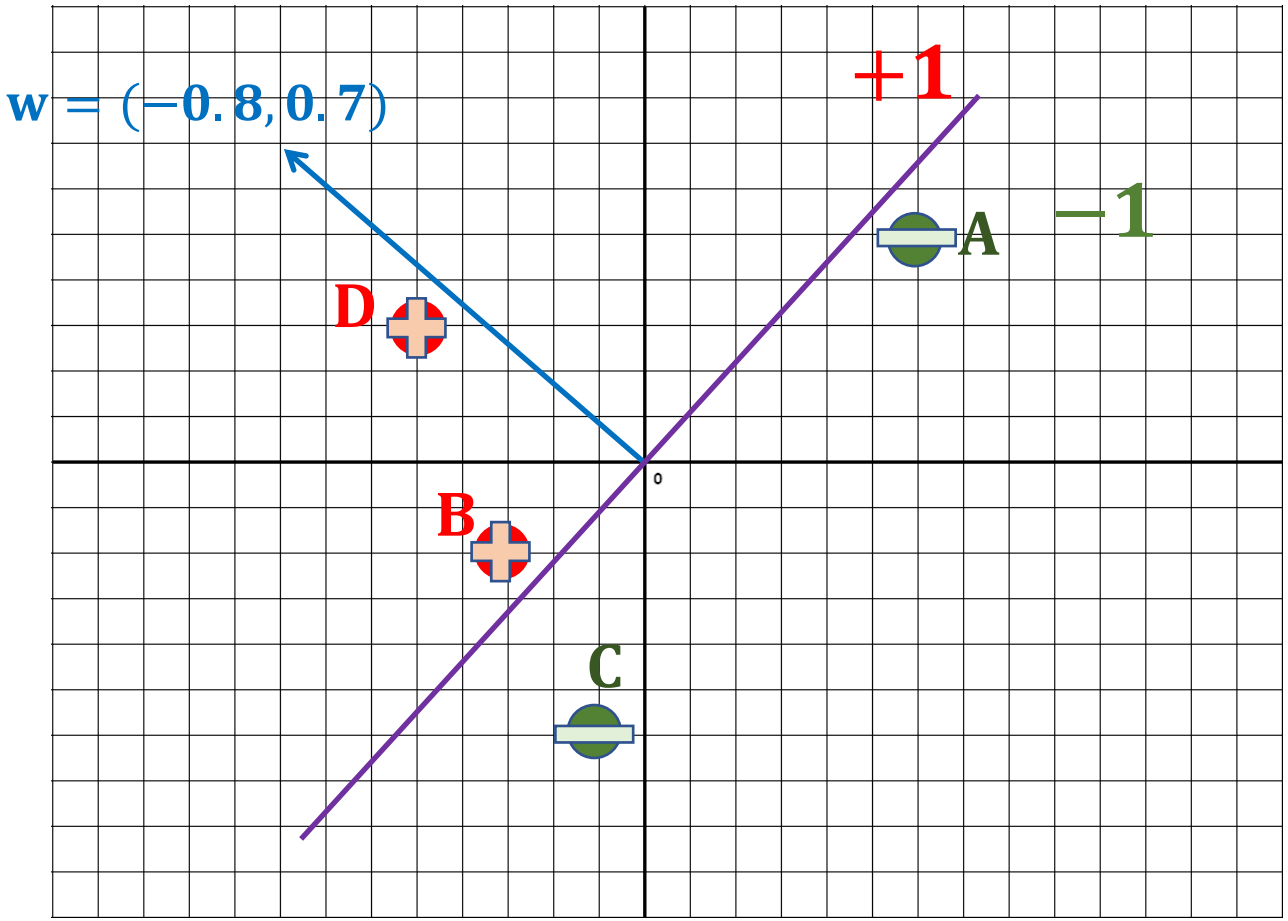
Training data

Item	x_1	x_2	Class
A	0.6	0.5	0
B	-0.3	-0.2	1
C	-0.1	-0.6	0
D	-0.5	0.3	1

For input A

- $\mathbf{w}^{\mathrm{T}}\mathbf{x} = -0.8 \cdot 0.6 + 0.7 \cdot 0.5 = -0.13 < 0$
- $y = 0$, and $t = 0$
- Don't update the weights
- **After this step, all data items have been correctly classified with the weights**

Learning rate: $\eta = 1$, $\mathbf{w} = (-0.8, 0.7)$



Important Remarks

- If data linearly separable, Perceptron is guaranteed to find a weight vector with 0 training error. However, may need many iterations over data to converge to such a weight vector

- Perceptron does not converge if the data is not linearly separable. In this case, it may cycle through some weight vectors without stopping

- Perceptron is sensitive to initialisation. Two runs of the algorithm may yield different weights if they visit the training examples in different orders.

Multiclass Perceptron

Note:

- y_n: predicted label of $\mathbf{x}_n$, t_n: true label of $\mathbf{x}_n$
- Maintain a set of k weight vectors, where k is the number of classes.

Algorithm

Repeat until all data points are classified correctly or the error $< \epsilon$:

- For each training point $\mathbf{x}_n$, perform the following
  - Predict the class whose weight vector produces the highest score: $y_n = \arg\max_k(\mathbf{w}_k^{\mathrm{T}}\,\mathbf{x}_n)$
  - If correct, do nothing
  - If wrong, update the weights (i.e., if $(y_n \neq t_n)$)
    - reduce score of wrong answer: $\mathbf{w}_{y_n} = \mathbf{w}_{y_n} - \eta\mathbf{x}_n$
    - increase score of right answer: $\mathbf{w}_{t_n} = \mathbf{w}_{t_n} + \eta\mathbf{x}_n$

Laboratory (Week 5)

- Practice how to generate synthetic data for binary classification problem
- Build your perceptron model based on the data

What will we learn in Week 6?

Learn Probabilistic Generative Models

Learn Probabilistic Discriminative Models

THANK YOU
for your attention!